

LAB 3: COMPONENT MODELLING & ARCHITECTURAL PATTERN SELECTION

Scenario: Self-Service Coffee Kiosk System

Selected Architecture: Layered Architecture

Architecture Comparison

Layered Architecture: Separates the presentation, business logic, and data/hardware responsibilities. This fits the kiosk's touchscreen interface, order and payment processing, menu/pricing storage, and receipt printing.

Microservices: Splits functionality into independently deployed services. This can help large, independently scaling systems, but adds deployment and communication overhead for a single kiosk.

Client-Server: Separates a client interface from server-side services. It is a useful interaction model, but by itself does not describe the internal separation of the kiosk's business logic and data/hardware components.

Reasons for Selecting Layered Architecture

1. Separation of responsibilities: The presentation layer handles touchscreen interactions, the business layer manages orders and payments, and the data/hardware layer handles menu and pricing data and receipt printing. This separation makes the system easier to understand and modify.

2. Maintainability and clear interfaces: Components communicate through defined interfaces, supporting the required workflow for selecting a coffee and size, paying by credit card, and printing a receipt. A change to one component can be made with less impact on the others when its interface remains stable.

Security Advantage

A dedicated Payment Service creates a separate boundary for payment processing. The system should avoid storing sensitive card details unnecessarily and should limit payment data access to the components that need it.

Performance Benefit

The ordering workflow follows a clear path between components through defined interfaces. This reduces unnecessary dependencies and keeps communication straightforward, although actual speed depends on implementation and the payment terminal/network.

Conclusion: Layered Architecture is a suitable choice for this kiosk because it organizes the required functions clearly, supports maintainability, and provides a practical boundary around payment processing.